



UNIWERSYTET ŚLĄSKI
W KATOWICACH

Wydział Nauk Ścisłych i Technicznych

Przedmiot

Algorytmika i zaawansowane struktury danych

Autor

Mateusz Zaniewski

Wykorzystane zadania:

Zadanie 61. Podzielność przez 11 (***)

Spis Treści

1.	Opis zadania	3
2.	Teoria.....	3
2.1	Złożoność obliczeniowa i jej znaczenie	3
2.2	Notacja Big O.....	4
2.3	Klasy złożoności.....	5
2.4	Algorytmy rekurencyjne.....	6
2.5	Operacje na łańcuchach znaków.....	7
3.	Rozwiązanie	9
3.1	Przypomnienie zasady i danych wejściowych	9
3.2	Implementacja algorytmu	9
3.3	Analiza zmienna po zmiennej.....	10
3.4	Analiza złożoności	11
3.5	Analiza rekurencji	11
3.6	Złożoność całkowita	12
4.	Wnioski	13
5.	Bibliografia.....	13

1. Opis zadania

Zadanie polega na analizie jak najdokładniejszej złożoności obliczeniowej algorytmu sprawdzającego czy dana liczba jest podzielna przez 11. Należy zaprezentować z użyciem notacji Big O jak efektywnie będzie działać ten algorytm w zależności od długości sprawdzanego ciągu znaków (cyfry).

Samo sprawdzenie podzielności przez 11 działa według prostej reguły: wystarczy zsumować cyfry stojące na miejscach nieparzystych, osobno zsumować cyfry na miejscach parzystych, a następnie sprawdzić, czy ich różnica jest podzielna przez 11.

Przykład z zadania:

Liczba 854073, cyfry na pozycjach nieparzystych (1., 3., 5.): 8, 4, 7 — ich suma to 19. Cyfry na pozycjach parzystych (2., 4., 6.): 5, 0, 3 — ich suma to 8. Różnica: $19 - 8 = 11$. Ponieważ 11 jest podzielne przez 11, liczba 854073 również jest podzielna przez 11.

Treść zadania:

Za pomocą notacji Big O() opisz najdokładniej jak potrafisz czas działania powyższego algorytmu w zależności od długości łańcucha n .

2. Teoria

2.1 Złożoność obliczeniowa i jej znaczenie

Złożoność obliczeniowa jest miarą zasobów (czasu i pamięci) niezbędnych do rozwiązania danego problemu algorytmem. Jest fundamentalnym narzędziem oceny efektywności algorytmów, pozwalającym na obiektywne porównywanie różnych podejść i przewidywanie zachowania programów dla dużych danych wejściowych ¹.

Złożoność czasowa określa, jak rośnie czas działania algorytmu wraz z rozmiarem danych wejściowych. Formalnie, jeżeli $T(n)$ oznacza liczbę operacji elementarnych wykonywanych przez algorytm dla danych rozmiaru n , to złożoność czasowa opisuje asymptotyczne zachowanie funkcji $T(n)$.

¹ T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Wprowadzenie do algorytmów, Wydawnictwo Naukowe PWN, Warszawa 2012, rozdział 3: Growth of Functions, s. 41-72.

Złożoność pamięciowa (ang. *space complexity*) opisuje ilość pamięci potrzebnej do wykonania algorytmu. Wyróżniamy pamięć pomocnicza (dodatkowa pamięć poza danymi wejściowymi) i pamięć całkowita.

Analizując złożoność obliczeniową, zazwyczaj rozpatrujemy trzy przypadki:

- Przypadek pesymistyczny (ang. *worst case*): maksymalna liczba operacji dla danych rozmiaru n . Najczęściej stosowany w praktyce.
- Przypadek średni (ang. *average case*): oczekiwana liczba operacji dla losowych danych rozmiaru n . Wymaga zdefiniowania rozkładu prawdopodobieństwa na danych.
- Przypadek optymistyczny (ang. *best case*): minimalna liczba operacji dla danych rozmiaru n . Rzadko użyteczny praktycznie.

Rozmiar danych wejściowych zależy od samego algorytmu i danych jakie przyjmuje. Dla algorytmów sortowania jest to liczba elementów, dla algorytmów grafowych — liczba wierzchołków i krawędzi, dla algorytmów łańcuchowych — długość łańcucha (liczba znaków). W opisywanym zadaniu n oznacza długość łańcucha znaków reprezentującego liczbę.

2.2 Notacja Big O

Notacja Big O służy do oceny wydajności algorytmu. Liczona jest dokładnie jak bardzo liczba operacji w algorytmie wzrasta wraz z wzrostem rozmiaru danych. Stałe mnożniki i drobne szczegóły są nieistotne — liczy się ogólny kształt wzrostu.

Ważna cecha notacji Big O jest to, że ignoruje stałe. Nie ma znaczenia, czy algorytm wykonuje 3 operacje na element, czy 7 — oba są $O(n)$. Stałe mogą się różnić między komputerami, kompilatorami i językami programowania, więc są abstrahowane.

Podobnie ignorowane są składniki niższego rzędu. Jeżeli algorytm wykonuje $n^2 + 5n + 100$ operacji, to dla dużych n dominuje n^2 — pozostałe składniki stają się coraz mniej ważne. Notacja wynosi wtedy $O(n^2)$.²

Przykład: Jeżeli algorytm przetwarza każdy element danych dokładnie raz (i w każdym kroku robi tyle samo pracy), to jest algorytmem liniowym — $O(n)$. Jeżeli dla 10 elementów wykonuje 10 operacji, to dla 1000 elementów wykona około 1000 operacji. Czas rośnie proporcjonalnie do ilości danych.

Notacja Big O opisuje górny pułap wzrostu — mówi: algorytm działa co najwyżej tak szybko, jak $g(n)$. Ale istnieją dwie pokrewne notacje, które dają pełniejszy obraz.

Notacja O (duże O) — górny limit. $O(n)$ oznacza: czas działania nie przekroczy stałej krotności n . Działa jak gwarancja maksymalna: najgorzej tyle to potrwa.

² A.Y.Bhargava Algorytmy, Wydawnictwo Helion, Gliwice 2017, rozdział 1: Wprowadzenie do algorytmów, s. 10-13

Notacja Omega — dolny limit. Omega(n) oznacza: czas działania wyniesie co najmniej tyle. Działa jak gwarancja minimalna: szybciej się nie da.

Notacja Theta — dokładna ocena. Theta(n) oznacza, że algorytm działa zarówno co najwyżej, jak i co najmniej liniowo — inaczej mówiąc, dokładnie liniowo. Jest to najsilniejsze i najbardziej precyzyjne twierdzenie o złożoności.³

2.3 Klasy złożoności

Różne algorytmy rosną w zupełnie różnym tempie. Wyobraźmy sobie, że $n = 1000$ (np. tysięcy cyfr w naszym łańcuchu). Poniżej pokazano, ile operacji wykona algorytm w zależności od swojej klasy złożoności:

Notacja	Nazwa	Liczba operacji dla $n = 1000$	Czy to mało?
$O(1)$	Stała	1	Tak – zawsze 1 krok
$O(\log n)$	Logarytmiczna	10	Znakomicie
$O(n)$	Liniowa	n	Bardzo dobrze
$O(n \log n)$	Liniowo-logarytmiczna	Ok. 10 000	Dobrze
$O(n^2)$	Kwadratowa	1 000 000	Dopuszczalnie
$O(2^n)$	Wykładnicza	2^{1000}	Praktycznie niemożliwa
$O(n!)$	Silniowa	1000!	Nie do wykonania

Tabela 1. Porównanie klas złożoności dla $n = 1000$

Warto intuitywnie rozumieć każdą z tych klas:

- Algorytm zawsze robi tyle samo pracy, niezależnie od rozmiaru danych. Przykładem jest sprawdzenie pierwszego elementu tablicy. Nawet gdyby tablica miała milion elementów, zagląda tylko do jednego miejsca. $O(1)$ — stała złożoność:
- Czas rośnie, ale bardzo powoli. Jeżeli dane rosną dziesięciokrotnie, czas rośnie zaledwie o jeden krok. Poszukiwanie binarne to klasyczny przykład: przy 1000 elementach wystarcza około 10 porównań, przy miliardzie — zaledwie 30. $O(\log n)$ — logarytmiczna:

³ D. E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, Addison-Wesley, Boston 1997, sekcja 1.2.11: Asymptotic representations, s. 107-116.

- Algorytm wykonuje stałą liczbę operacji dla każdego elementu danych. Jeżeli danych jest dwa razy więcej, czas wykonania jest dwa razy dłuższy. Nasz algorytm podzielności przez 11 jest właśnie tutaj. $O(n)$ — liniowa:
- Dane dwa razy większe = czas cztery razy dłuższy. Tak działa np. porównywanie każdego elementu z każdym innym, jak w bąbelkowym sortowaniu. $O(n^2)$ — kwadratowa:
- Czas rośnie tak szybko, że już dla $n = 100$ obliczenia są praktycznie nie do wykonania na żadnym komputerze. $O(2^n)$ i $O(n!)$ — wykładnicza i silniowa:

Warto też wiedzieć, że algorytmy możemy porównywać według ich przynależności do szerokich klas złożoności. Najważniejsza z nich to klasa P — zbiór algorytmów, które działają w czasie wielomianowym (np. $O(n)$, $O(n^2)$, $O(n^3)$). Problemy w klasie P są uznawane za *rozwiązywalne w rozsądnym czasie*. Nasza podzielność przez 11 jest w klasie P — co więcej, jest w podklasie L, czyli algorytmów, które zużywają bardzo mało pamięci roboczej.⁴

2.4 Algorytmy rekurencyjne

Niektóre algorytmy rozwiązują problem w ten sposób, że rozbijają go na mniejsze problemy tego samego rodzaju, a następnie łączą wyniki. Takie podejście nazywa się rekurencja (ang. *recursion*). Funkcja rekurencyjna wywołuje sama siebie — ale z mniejszymi danymi — aż dotrze do tak prostego przypadku, że może go rozwiązać bezpośrednio [4].

Do analizy złożoności algorytmów rekurencyjnych służy specjalne narzędzie matematyczne — twierdzenie o rekurencji głównej (ang. *Master Theorem*). Opisuje ono algorytmy, które dzielą swój problem na a podproblemy, każdy o rozmiarze n/b , i potrzebują jeszcze $f(n)$ pracy do scalenia wyników. Wzór wygląda tak:

$$T(n) = a \cdot T(n/b) + f(n)$$

Gdzie:

- $T(n)$ — czas działania
- a — liczba podproblemów
- $T(n/b)$ — rozmiar podproblemu
 - b — współczynnik podziału
- $f(n)$ — praca poza rekurencją

Twierdzenie mówi, że wynik (złożoność) zależy od tego, czy praca scalania $f(n)$ jest mała, równa, czy duża względem pracy podziałów

⁴ C. H. Papadimitriou, Computational Complexity, Addison-Wesley, Reading MA 1994, rozdział 2: The class P, s. 14-68.

Popularny przykład: Mergesort (sortowanie przez scalanie) dzieli tablice na dwa równe kawałki ($a=2$, $b=2$), a łączenie zajmuje $O(n)$ pracy. Twierdzenie daje wynik $\Theta(n \log n)$ — i tak właśnie działa Mergesort, co zgadza się z intuicją.

Aby poprawnie analizować algorytm rekurencyjny, należy zwrócić uwagę na trzy elementy. Pierwszym jest przypadek bazowy (ang. *base case*), czyli warunek, po spełnieniu którego funkcja przestaje wywoływać samą siebie i zwraca gotowy wynik. Drugim jest krok rekurencyjny, w którym problem zostaje sprowadzony do mniejszej wersji samego siebie. Trzecim — głębokość rekurencji, czyli liczba zagnieżdżonych wywołań, zanim dotrzemy do przypadku bazowego. Brak poprawnego przypadku bazowego prowadzi do rekurencji nieskończonej, a zbyt duża głębokość — do przepełnienia stosu (ang. *stack overflow*).

Każde wywołanie funkcji rekurencyjnej zajmuje miejsce na tzw. stosie wywołań (ang. *call stack*). Z tego powodu algorytm rekurencyjny zużywa pamięć proporcjonalną do głębokości rekurencji — nawet jeśli sama praca obliczeniowa na pojedynczym poziomie jest niewielka. Głębokość rekurencji jest więc równie ważna jak liczba operacji, ponieważ decyduje o złożoności pamięciowej algorytmu.

To, jak szybko maleje rozmiar podproblemu, decyduje o całej złożoności. Jeżeli rozmiar maleje o stałą wartość (z n do $n - 1$), rekurencja ma zwykle głębokość liniową $O(n)$. Jeżeli maleje o stały czynnik (z n do $n/2$), głębokość jest logarytmiczna $O(\log n)$. Istnieją jednak algorytmy, w których rozmiar danych kurczy się znacznie gwałtowniej — na przykład z n do $\log n$. Wówczas głębokość rekurencji opisuje funkcja logarytmu iterowanego $\log^*(n)$, która rośnie tak wolno, że dla wszystkich praktycznych danych przyjmuje wartość nie większą niż 5. Z taką właśnie, nietypową sytuacją mamy do czynienia w analizowanym zadaniu.

Warto podkreślić, że twierdzenie o rekurencji głównej stosuje się wyłącznie wtedy, gdy podproblemy mają rozmiar n/b , czyli stanowią stały ułamek pierwotnego problemu. Jeżeli rozmiar maleje szybciej (np. z n do $\log n$) albo wolniej (z n do $n - 1$), twierdzenie nie ma zastosowania — złożoność trzeba wtedy wyznaczyć bezpośrednio, rozwijając rekurencję krok po kroku i sumując pracę wykonaną na wszystkich poziomach. Metodę tę nazywa się metodą podstawiania (ang. *substitution method*) lub metodą drzewa rekurencji.

2.5 Operacje na łańcuchach znaków

W naszym algorytmie dane wejściowe nie są liczbą w sensie matematycznym — są łańcuchem znaków (ang. *string*), czyli ciągiem cyfr zapisanych jako tekst: np. "854073". Dlaczego? Bo zakładamy, że nie wiemy z góry, jak długa jest liczba — może mieć 5 cyfr, może mieć 500. Łańcuch to elastyczny sposób przechowywania ciągu cyfr o dowolnej długości.

Różne operacje na łańcuchach mają różne koszty. Poniżej przedstawiono te, które są istotne dla naszego algorytmu⁵:

Operacja	Koszt	Co to oznacza w praktyce
Odczytanie długości (string.length)	$O(1)$	Natychmiastowe — długość jest zapamiętana
Dostęp do pojedynczego znaku string[i]	$O(1)$	Jak szufladka w szafie — od razu tam trafiamy
Przejsięcie przez cały łańcuch (pętla)	$O(n)$	Musimy zajrzeć do każdej szufladki raz
Łączenie dwóch łańcuchów	$O(n+m)$	Trzeba skopiować oba — proporcjonalnie do ich długości
Porównanie dwóch łańcuchów	$O(\min(n,m))$	Sprawdzamy znak po znaku, aż natrafimy na różnice

Tabela 2. Koszt typowych operacji na łańcuchach znaków

Kluczowy wniosek: jeżeli pętla przechodzi przez cały łańcuch raz, a w każdym kroku robi stała ilość pracy (np. jedno dodawanie), to całkowity czas to $O(n)$.

⁵ D. Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press, Cambridge 1997, rozdział 1: Exact String Matching: The Fundamental String Problem, s. 2-15.

3. Rozwiązanie

3.1 Przypomnienie zasady i danych wejściowych

Analizowany algorytm sprawdza, czy liczba zapisana jako łańcuch cyfr jest podzielna przez 11. Wykorzystuje przy tym cechę podzielności: liczba jest podzielna przez 11, jeżeli różnica między sumą cyfr na pozycjach parzystych a sumą cyfr na pozycjach nieparzystych jest podzielna przez 11. Algorytm stosuje tę regułę rekurencyjnie — z liczby $abs(s_2 - s_1)$ tworzy nowy, krótszy łańcuch i powtarza obliczenie aż do uzyskania pojedynczej cyfry. Jeżeli ta cyfra to 0, liczba jest podzielna przez 11 (wynik *TAK*), w przeciwnym razie — nie (wynik *NIE*). Dla przykładu z zadania liczba 854073 redukuje się do łańcucha „11”, ten do „0”, więc wynikiem jest *TAK*. Rozmiarem danych wejściowych jest n — długość łańcucha, czyli liczba cyfr.

3.2 Implementacja algorytmu

Pseudokod z zadania zaimplementowano w języku JavaScript. Zmienne s_1 i s_2 z pseudokodu odpowiadają tu zmiennym nieparzyste oraz parzyste. Indeksowanie łańcucha w JavaScript zaczyna się od zera, dlatego pozycji i (liczonej od 1) odpowiada znak `string[i - 1]`.

```

1  ✓ function podzielna11(string) {
2      const strLength = string.length;
3      if (strLength === 1) {
4          return string[0] === '0' ? 'TAK' : 'NIE';
5      }
6      let nieparzyste = 0;
7      let parzyste = 0;
8      for (let i = 1; i <= strLength; i++) {
9          const digit = parseInt(string[i - 1]);
10         if (i % 2 === 0) {
11             parzyste += digit;
12         } else {
13             nieparzyste += digit;
14         }
15     }
16     const newString = Math.abs(parzyste - nieparzyste).toString();
17     return podzielna11(newString);
18 }

```

Rysunek 1 Implementacja algorytmu podzielna11 w języku Javascript

3.3 Analiza zmienna po zmiennej

Przed wyznaczeniem złożoności warto prześledzić rolę każdej zmiennej, ponieważ to ich zakres wartości decyduje o koszcie pojedynczych operacji oraz o tym, jak szybko kurczy się problem przy każdym wywołaniu rekurencyjnym.

Zmienna	Pseudokod	Znaczenie	Rozmiar / zakres
string	x	Analizowany łańcuch cyfr przekazany do funkcji.	n znaków
strLength	n	Długość łańcucha, czyli liczba cyfr.	liczba całkowita
nieparzyste	s1	Suma cyfr stojących na pozycjach nieparzystych.	od 0 do $9 \cdot \lfloor n/2 \rfloor$
parzyste	s2	Suma cyfr stojących na pozycjach parzystych.	od 0 do $9 \cdot \lfloor n/2 \rfloor$
i	i	Licznik pętli — numer aktualnie czytanej pozycji.	od 1 do n
digit	x[i]	Bieżąca cyfra po konwersji znaku na liczbę.	od 0 do 9
newString	x (nowe)	Łańcuch reprezentujący wartość $\text{abs}(s2 - s1)$.	$O(\log n)$ znaków

Tabela 3. Zmienne wykorzystywane przez algorytm i ich charakterystyka.

Kluczowa obserwacja dotyczy zmiennych nieparzyste i parzyste. Każda z nich jest sumą najwyżej n cyfr, a pojedyncza cyfra nie przekracza 9, więc każda z sum jest nie większa niż $9n$. W konsekwencji wartość $\text{abs}(s2 - s1)$ również nie przekracza $9n$, a jej zapis dziesiętny

ma jedynie $\lceil \log_{10}(9n) \rceil + 1 = O(\log n)$ cyfr. To właśnie dlatego łańcuch przekazywany do kolejnego wywołania jest dramatycznie krótszy od pierwotnego — i to ta zależność, a nie sama liczba wywołań, kształtuje złożoność algorytmu.

3.4 Analiza złożoności

Poniższa tabela przypisuje koszt każdej linii implementacji w obrębie jednego wywołania funkcji (bez uwzględnienia wywołania rekurencyjnego z linii 17, które analizujemy osobno). Zapis „ $O(1) \times n$ ” oznacza operację o stałym koszcie wykonywaną w każdej z n iteracji pętli.

Linia	Operacja	Koszt	Uzasadnienie
1	<code>const strLength = string.length</code>	$O(1)$	Odczyt zapamiętanej długości łańcucha (por. Tabela 2).
2	<code>if (strLength === 1)</code>	$O(1)$	Pojedyncze porównanie liczb całkowitych.
3	<code>return string[0] === '0' ? ...</code>	$O(1)$	Dostęp do jednego znaku i porównanie; tylko w przypadku bazowym.
4	<code>let nieparzyste = 0; let parzyste = 0</code>	$O(1)$	Inicjalizacja dwóch zmiennych liczbowych.
5	<code>for (i = 1; i <= strLength; i++)</code>	$O(n)$	Pętla wykonuje się n razy; sam nagłówek to $O(1)$ na iterację.
6	<code>const digit = parseInt(string[i-1])</code>	$O(1) \times n$	Dostęp do znaku i konwersja jednej cyfry w każdej iteracji.
7	<code>if (i % 2 === 0)</code>	$O(1) \times n$	Sprawdzenie parzystości pozycji w każdej iteracji.
8–9	<code>parzyste += digit / nieparzyste += digit</code>	$O(1) \times n$	Jedno dodawanie liczb w każdej iteracji.
10	<code>Math.abs(parzyste - nieparzyste).toString()</code>	$O(\log n)$	Odejmowanie to $O(1)$; zamiana liczby $\leq 9n$ na łańcuch daje $O(\log n)$ cyfr.
11	<code>return podzieln11(newString)</code>	$T(O(\log n))$	Wywołanie rekurencyjne na łańcuchu długości $O(\log n)$.

Tabela 4. Koszt poszczególnych linii algorytmu w obrębie jednego wywołania.

Sumując koszty w obrębie jednego wywołania, otrzymujemy: trzy operacje stałe (linie 1–4), pętlę o koszcie $O(n)$ (linie 5–9) oraz utworzenie nowego łańcucha o koszcie $O(\log n)$ (linia 10). Ponieważ dla dużych n składnik liniowy dominuje nad logarytmicznym, koszt pojedynczego wywołania (z pominięciem rekurencji) wynosi:

$$T(n) = O(1) + O(n) + O(\log n) = O(n).$$

3.5 Analiza rekurencji

Linia 11 wywołuje funkcję ponownie, tym razem na łańcuchu o długości $O(\log n)$. Prowadzi to do następującej zależności rekurencyjnej, w której c jest pewną stałą, a n — długością bieżącego łańcucha:

$$T(n) = c \cdot n + T(n'), \text{ gdzie } n' = O(\log n), \text{ oraz } T(1) = O(1).$$

Jak zauważono w części teoretycznej, twierdzenie o rekurencji głównej tu nie obowiązuje — podproblem nie ma rozmiaru n/b (stałego ułamka), lecz $\log n$, a więc kurczy się znacznie szybciej. Złożoność wyznaczamy więc metodą podstawiania, rozwijając rekurencję poziom po poziomie i sumując pracę:

$$T(n) = c \cdot n + c \cdot \log n + c \cdot \log \log n + \dots + O(1).$$

Składniki tego ciągu maleją błyskawicznie: po pierwszym kroku rozmiar spada z n do $\log n$, po drugim do $\log \log n$ i tak dalej. Ich suma jest zdominowana przez pierwszy, liniowy wyraz, dlatego całość zachowuje się jak $O(n)$. Liczbę poziomów (głębokość rekurencji) opisuje logarytm iterowany $\log^*(n)$, który dla każdej realnej danej nie przekracza 5 — rekurencja jest więc praktycznie płytka.

Najlepiej widać to na konkretnym przykładzie. Dla łańcucha o długości $n = 1000$ kolejne wywołania kurczą problem w następujący sposób:

Poziom rekurencji	Maks. liczba cyfr na wejściu	Maks. wartość $\text{abs}(s_2 - s_1)$	Praca na poziomie
0	1000	—	≈ 1000
1	4	≤ 4500	≈ 4
2	2	≤ 18	≈ 2
3	1	≤ 9	przypadek bazowy

Tabela 5. Szybkość kurczenia się problemu dla wejścia o długości $n = 1000$.

Już po pierwszym wywołaniu z 1000 cyfr zostają najwyżej 4, a cała praca poza pierwszym poziomem to zaledwie kilka operacji. Sumaryczny koszt to około $1000 + 4 + 2 \approx 1006$ operacji, czyli w praktyce tyle, ile zajmuje jednokrotne przejście przez dane wejściowe.

3.6 Złożoność całkowita

Łącząc analizę pojedynczego wywołania z analizą rekurencji, otrzymujemy całkowitą złożoność czasową algorytmu:

$$T(n) = O(n) \quad \text{—} \quad \text{złożoność liniowa.}$$

Co więcej, jest to ocena dokładna w sensie notacji Theta, czyli $\Theta(n)$. Dolne ograniczenie $\Omega(n)$ wynika z tego, że już pierwsze wywołanie musi odczytać każdą z n cyfr przynajmniej raz — szybciej problemu rozwiązać się nie da. Ponieważ górne ograniczenie również wynosi $O(n)$, obie granice się spotykają i otrzymujemy $O(n)$. Złożoność jest taka sama w przypadku optymistycznym, średnim i pesymistycznym, gdyż liczba odczytanych cyfr nie zależy od konkretnych wartości w łańcuchu.

Dodatkowo warto odnotować złożoność pamięciową. Algorytm tworzy nowy łańcuch o długości $O(\log n)$, a głębokość stosu wywołań wynosi $\log^*(n)$ — wartość praktycznie stałą. Pamięć pomocnicza jest więc rzędu $O(\log n)$, czyli podliniowa. Potwierdza to wcześniejszą uwagę, że problem należy nie tylko do klasy P, ale i do oszczędnej pamięciowo podklasy L.

4. Wnioski

Czas działania algorytmu podzielna11 w zależności od długości łańcucha n wynosi $\Theta(n)$ — jest **liniowy**.

Choć algorytm jest rekurencyjny, rekurencja nie zwiększa jego rzędu złożoności powyżej liniowego. Decyduje o tym jedna obserwacja: pojedyncze przejście przez łańcuch zamienia n cyfr na liczbę nieprzekraczającą $9n$, której zapis ma już tylko $O(\log n)$ cyfr. Problem kurczy się więc z n do $\log n$ w jednym kroku, a praca na wszystkich kolejnych poziomach ($\log n + \log \log n + \dots$) jest pomijalna wobec pierwszego, liniowego przejścia. Głębokość rekurencji opisuje logarytm iterowany $\log^*(n)$, który dla wszelkich realnych danych nie przekracza 5.

Z analizy płynie też wniosek metodologiczny: nie do każdej rekurencji da się zastosować twierdzenie o rekurencji głównej. Wymaga ono podziału na podproblemy rozmiaru n/b , podczas gdy tutaj rozmiar maleje aż do $\log n$. Właściwym narzędziem okazała się metoda podstawiania — bezpośrednie rozwinięcie rekurencji i zsumowanie pracy z poszczególnych poziomów.

W ujęciu praktycznym oznacza to, że algorytm jest bardzo wydajny: poradzi sobie nawet z liczbami o milionach cyfr w czasie proporcjonalnym do ich długości, zużywając przy tym jedynie podliniową ($O(\log n)$) pamięć pomocniczą. Należy do klasy P problemów rozwiązywalnych w rozsądnym czasie, a ze względu na oszczędność pamięci — do podklasy L.

5. Bibliografia

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Wprowadzenie do algorytmów, Wydawnictwo Naukowe PWN, Warszawa 2012, rozdział 3: Growth of Functions, s. 41-72.
2. A.Y.Bhargava Algorytmy, Wydawnictwo Helion, Gliwice 2017, rozdział 1: Wprowadzenie do algorytmów, s. 10-13
3. D. E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, Addison-Wesley, Boston 1997, sekcja 1.2.11: Asymptotic representations, s. 107-116.
4. C. H. Papadimitriou, Computational Complexity, Addison-Wesley, Reading MA 1994, rozdział 2: The class P, s. 14-68.

5. D. Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press, Cambridge 1997, rozdział 1: Exact String Matching: The Fundamental String Problem, s. 2-15.